D&D SIMULATOR — APPLICATION UI IMPLEMENTATION INSTRUCTIONS

ROLE

Act as the lead desktop UI architect and implementation engineer for the D&D Simulator project.

Your job is to recreate the approved Version 1 interface as a functional desktop application UI while keeping the entire visual system modular enough to support multiple themes later.

The selected Version 1 concept image is the primary visual reference for layout, proportions, density, information hierarchy, and overall user experience.

The initial theme is High Fantasy.

Future themes must include:

1. High Fantasy
2. Old Scroll
3. Modern
4. Dark Fantasy
5. Additional custom themes added later without rebuilding the interface

Do not copy copyrighted Old School RuneScape artwork, icons, textures, logos, or exact interface assets. The interface may have a classic compact fantasy-RPG and OSRS-inspired structure, but all assets and styling must be original.

==========

1. CORE UI OBJECTIVE

==========

Build a desktop interface that feels like a complete fantasy adventure control center.

The player must be able to access the following systems directly from the main screen:

- Character profile
- Health and resources
- Inventory
- Equipment
- Skills
- Quests
- Objectives
- Map
- Journal
- Recent events
- NPC information
- Relationships
- Spellbook or abilities
- Game settings
- Save and load controls
- Main narrative interaction area

The player should not need to open multiple separate windows for ordinary play.

The main screen must feel dense and useful without feeling disorganized.

The application must visually resemble the approved Version 1 layout:

- Large narrative and interaction area on the left and center
- Persistent utility sidebar on the right
- Character summary across the upper-left area
- Location and world information across the upper-center area
- Map at the top of the right sidebar
- Skills and inventory visible or one click away
- Quest and objective information visible in the central area
- Event log and navigation controls along the bottom

==========

2. TECHNOLOGY TARGET

==========

Use the existing project technology decisions.

Preferred initial implementation:

- Python
- PySide6 / Qt
- SQLite-backed application state
- Pydantic or typed application models
- QSS for styling
- SVG or transparent PNG assets for icons and decorative elements

Do not change the technology stack without first explaining why and receiving approval.

The UI must read factual values from application services or mock data providers.

Do not call the LLM merely to display:

- Inventory
- Skills
- Character statistics
- Quest status
- Equipment
- Currency
- Health
- Map locations
- Saved data

The LLM may later provide narration, dialogue, descriptions, summaries, and proposed actions, but the application remains the source of truth.

==========

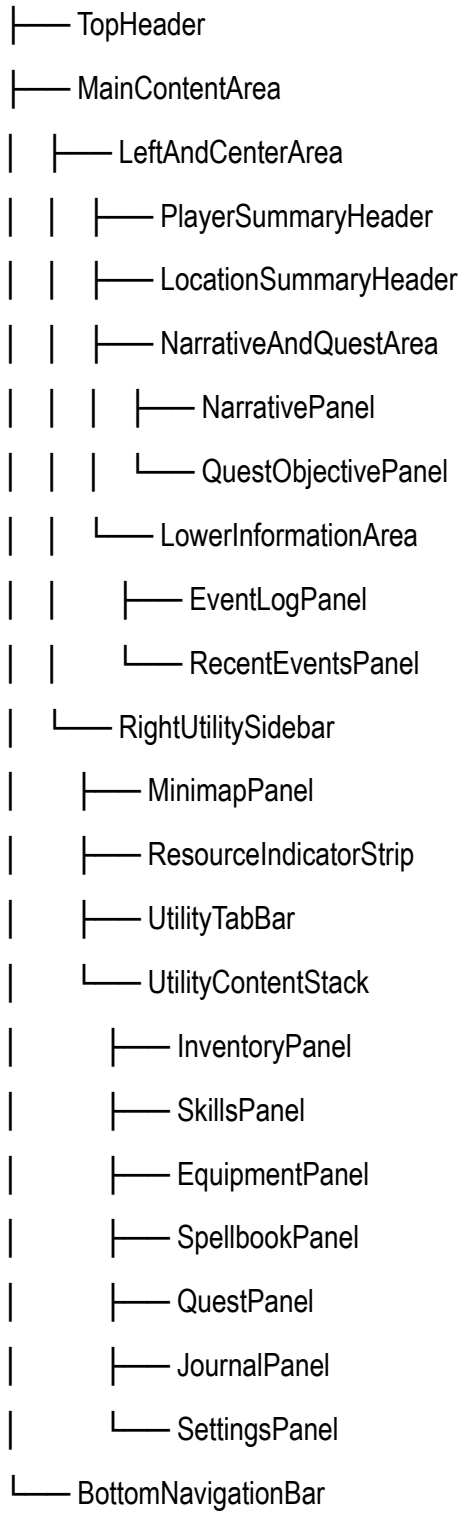
3. LAYOUT ARCHITECTURE

==========

Build the main window using reusable layout regions.

Use a structure similar to:

MainWindow



The exact implementation may use:

- QMainWindow
- QSplitter
- QStackedWidget
- QTabBar
- QScrollArea
- QFrame
- QGridLayout
- QVBoxLayout
- QHBoxLayout

Prefer reusable custom widgets instead of placing all behavior directly in MainWindow.

==========

4. APPROVED VERSION 1 SCREEN STRUCTURE

==========

Use the following approximate screen proportions for a standard 16:9 desktop window.

A. RIGHT UTILITY SIDEBAR

Width:

- Approximately 28–32% of the total application width
- Minimum practical width around 360 pixels
- Must remain usable when the application is resized

Contents:

1. Circular or framed minimap at the top
2. Vertical or compact resource indicators beside or beneath the minimap
3. Icon-based tab row
4. Skills list or grid
5. Inventory grid
6. Bottom utility icon row

The sidebar should feel persistent, similar to a classic fantasy RPG control panel.

B. LEFT AND CENTER CONTENT REGION

Width:

- Approximately 68–72% of the application

Top area:

- Character portrait
- Character name
- Level and class
- Health and secondary resources
- Experience progress
- Currency
- Carry weight
- Current location
- Time of day

- Weather
- Small location illustration or visual preview

Middle area:

- Large narrative panel
- Active objective or quest panel beside it
- Action input field at the bottom of the narrative panel

Lower area:

- Event log with filter tabs
- Recent events or timeline
- Main navigation buttons

C. BOTTOM NAVIGATION

Include clear navigation for:

- Profile
- Inventory
- Skills
- Quests
- Map
- Journal
- Settings

Opening one of these should update the main utility content area or open an appropriate in-application panel.

Avoid spawning unnecessary external windows.

==========

5. PLAYER SUMMARY HEADER

==========

Create a compact but highly readable player summary component.

Required information:

- Character portrait
- Character name
- Level
- Class or archetype
- Current health / maximum health
- Secondary resource such as mana, stamina, focus, or spell power
- Experience progress
- Gold
- Carry weight or inventory capacity

Design requirements:

- Portrait displayed inside a themed decorative frame
- Health uses a clearly recognizable red progress bar

- Mana or magic uses blue

- Stamina may use green or yellow

- Experience uses a separate progress indicator

- Resource values must remain readable when bars are nearly empty

- Tooltip support for every resource

- Clicking the portrait or name opens the full Profile panel

Do not hard-code resource types into the visual structure.

The widget should accept a configurable list of resources so different campaigns or classes can use different resource systems.

==========

6. LOCATION SUMMARY HEADER

==========

Display:

- Current location name

- Region

- Time

- Day

- Weather

- Optional danger level

- Optional location image

- Optional travel or route status

The location visual should be treated as application content, not as part of the theme.

Theme assets and campaign visual assets must remain separate.

Example distinction:

Theme asset:

- Stone frame around the location preview

Campaign asset:

- Illustration of Greystone Road

==========

7. NARRATIVE PANEL

==========

The narrative panel is the primary play area.

Required features:

- Scrollable rich-text narrative history
- Distinct formatting for narration, dialogue, system results, quest updates, combat messages, and errors
- Text input field
- Send or action button
- Optional suggested-action buttons
- Keyboard shortcut support
- Streaming-text support for future LM Studio integration

- Copy-text support
- Search or history support later

Message styles should be semantic rather than hard-coded.

Use message categories such as:

- narrative
- dialogue
- player_action
- system
- combat
- success
- warning
- error
- quest_update
- loot
- experience
- relationship_change

The theme determines how each category looks.

The narrative data must not contain theme-specific colors.

Example:

Correct:

message.category = "quest_update"

Incorrect:

message.color = "#4dff00"

==========

8. QUEST AND OBJECTIVE PANEL

==========

Display the active quest and current objectives without requiring the player to leave the main screen.

Required information:

- Active quest title
- Main objective
- Optional objectives
- Completed objectives
- Known clues
- Progress counts
- Deadlines
- Quest status
- Button to open full quest details

Use recognizable states:

- Active
- Complete

- Failed
- Optional
- Hidden or undiscovered
- Blocked

Completed objectives should be visually distinct but remain readable.

Do not expose hidden quest information from the database.

The application service decides what is visible.

==========

9. EVENT LOG

==========

Create a lower event log with filter tabs.

Recommended filters:

- All
- Combat
- System
- Quests
- NPCs
- Loot
- Travel

Each entry may include:

- Timestamp
- Turn number
- Small icon
- Event category
- Short description
- Clickable entity references where practical

The event log should be optimized for quick scanning rather than long-form reading.

The full narrative belongs in the narrative panel.

==========

10. RECENT EVENTS PANEL

==========

Create a compact timeline beside the event log.

Show examples such as:

- Arrived at a location
- Spoke with an NPC
- Completed a quest objective
- Defeated an enemy
- Discovered a location
- Rested

- Gained an item

Entries should use small original icons and compact text.

Clicking an entry may later open the relevant journal record, quest, NPC, or location.

==========

11. MINIMAP

==========

The minimap panel should support:

- Circular or ornamental framed presentation
- Player marker
- Quest marker
- NPC marker
- Location marker
- Point-of-interest marker
- Zoom controls
- Open-full-map control
- Optional compass
- Optional fog-of-war

The minimap itself must be data-driven.

The decorative frame belongs to the theme.

Do not merge map rendering logic with decorative styling.

==========

12. UTILITY TAB SYSTEM

==========

The right sidebar must use a reusable utility navigation system.

Recommended tabs:

- Combat
- Skills
- Abilities or Spells
- Inventory
- Equipment
- Character
- Quests
- Journal

Use icons plus tooltips.

The selected tab must be clear.

The utility content area should use QStackedWidget or an equivalent component so only the relevant panel is visible.

Tab behavior must remain identical across every theme.

Only the visual presentation changes.

==========

13. INVENTORY PANEL

==========

Create an OSRS-inspired but original inventory grid.

Required features:

- Configurable row and column count
- Item icon
- Stack quantity
- Durability or condition indicator where relevant
- Rarity border or marker
- Quest-item marker
- Selected-item state
- Hover tooltip
- Right-click context menu
- Drag support prepared for later use
- Capacity and weight display
- Currency display where appropriate

Actions may include:

- Use

- Equip
- Unequip
- Inspect
- Drop
- Split stack
- Assign to quick slot
- Compare
- Move

Do not perform item changes directly inside the widget.

The widget emits an action request to an inventory service or controller.

The service validates and commits the change.

==========

14. SKILLS PANEL

==========

The skills panel should support both:

- Compact OSRS-like icon and level grid
- Expanded named skill list

Required data:

- Skill icon

- Skill name
- Current level
- Current XP
- XP needed for next level
- Temporary bonuses or penalties
- Locked status
- Recently increased status

Allow the layout mode to be configured.

Do not assume all campaigns use the exact RuneScape skill list.

Skills must be loaded from campaign or rules data.

==========

15. EQUIPMENT PANEL

==========

Include equipment slots such as:

- Main hand
- Off hand
- Head
- Body
- Hands
- Legs
- Feet

- Neck
- Ring
- Trinket
- Ammunition
- Optional class-specific slots

Display:

- Equipped icon
- Empty slot state
- Comparison tooltip
- Derived-stat change preview
- Condition
- Requirements
- Locked slot state

The application validates equipment changes.

==========

16. PROFILE PANEL

==========

The expanded Profile panel should include:

- Portrait
- Identity
- Appearance

- Level
- Class
- Ancestry
- Background
- Attributes
- Derived statistics
- Conditions
- Injuries
- Resistances
- Reputation
- Progression
- Biography or character notes

The compact header should show only the information needed during ordinary play.

==========

17. VISUAL STYLE — INITIAL HIGH FANTASY THEME

==========

The first theme should closely follow the approved Version 1 concept.

Use original visual assets based on:

- Dark carved stone
- Aged dark wood
- Weathered brass or bronze
- Warm parchment

- Subtle gold accents
- Gemstone resource icons
- Decorative beveled panel edges
- Fantasy manuscript-inspired headings
- Highly readable serif or humanist body text
- Soft internal shadows
- Restrained highlights
- Slight texture without reducing readability

The interface should feel crafted and physical, but not excessively decorative.

Avoid:

- Overly bright gold
- Excessive gradients
- Tiny unreadable text
- Heavy textures behind paragraphs
- Decorative frames that waste large amounts of space
- Animation that distracts from reading
- Exact copies of OSRS icons or frame designs

==========

18. MULTI-THEME ARCHITECTURE

==========

This requirement is mandatory.

The interface must be built so future themes can be applied without rebuilding layouts or rewriting widget logic.

Separate the following layers:

1. Application logic
2. Widget structure
3. Layout geometry
4. Theme tokens
5. Theme-specific QSS
6. Theme-specific assets
7. Campaign artwork
8. User preferences

No widget should contain hard-coded theme colors, textures, fonts, border images, or decorative icons unless unavoidable.

Use semantic object names and dynamic properties.

Examples:

- objectName="playerSummaryPanel"
- objectName="narrativePanel"
- objectName="inventorySlot"
- property("state") = "selected"
- property("rarity") = "rare"
- property("messageType") = "quest_update"
- property("resourceType") = "health"

The stylesheet should target those semantic names and properties.

==========

19. THEME TOKEN SYSTEM

==========

Create a theme token system.

Recommended token categories:

COLORS

- background_primary
- background_secondary
- background_elevated
- panel_surface
- panel_surface_alt
- text_primary
- text_secondary
- text_muted
- accent_primary
- accent_secondary
- border_primary
- border_highlight
- success
- warning

- danger
- quest
- loot
- experience
- dialogue
- selection
- hover

TYPOGRAPHY

- font_display
- font_heading
- font_body
- font_monospace
- size_title
- size_heading_1
- size_heading_2
- size_body
- size_small
- weight_normal
- weight_bold

SPACING

- space_xs
- space_sm
- space_md

- space_lg
- space_xl

GEOMETRY

- border_radius_small
- border_radius_medium
- border_width
- panel_padding
- slot_size
- icon_size_small
- icon_size_medium
- icon_size_large

EFFECTS

- shadow_small
- shadow_medium
- hover_brightness
- disabled_opacity
- texture_opacity
- animation_fast
- animation_normal

ASSETS

- panel_frame

- header_frame
- button_frame
- minimap_frame
- inventory_slot_background
- selected_slot_overlay
- divider_horizontal
- divider_vertical
- resource_health_icon
- resource_mana_icon
- resource_stamina_icon

==========

20. THEME FILE FORMAT

==========

Use a clear theme folder structure.

Suggested structure:

```
ui/
├── main_window.py
├── controllers/
├── models/
├── services/
├── layouts/
│   └── main_layout.py
├── widgets/
```

```
│   ├── player_summary.py
│   ├── location_summary.py
│   ├── narrative_panel.py
│   ├── quest_objective_panel.py
│   ├── event_log_panel.py
│   ├── recent_events_panel.py
│   ├── minimap_panel.py
│   ├── resource_indicator.py
│   ├── utility_tab_bar.py
│   ├── inventory_grid.py
│   ├── skill_grid.py
│   ├── equipment_panel.py
│   └── themed_frame.py
├── panels/
│   ├── profile_panel.py
│   ├── inventory_panel.py
│   ├── skills_panel.py
│   ├── equipment_panel.py
│   ├── quests_panel.py
│   ├── map_panel.py
│   ├── journal_panel.py
│   └── settings_panel.py
└── themes/
    ├── theme_manager.py
    ├── theme_schema.py
    ├── high_fantasy/
    │   ├── theme.json
```

```
│   ├── theme.qss
│   ├── icons/
│   ├── frames/
│   ├── textures/
│   └── preview.png
├── old_scroll/
│   ├── theme.json
│   ├── theme.qss
│   └── assets/
├── modern/
│   ├── theme.json
│   ├── theme.qss
│   └── assets/
└── dark_fantasy/
    ├── theme.json
    ├── theme.qss
    └── assets/
```

ThemeManager responsibilities:

- Discover installed themes
- Validate theme metadata
- Load theme tokens
- Resolve asset paths
- Compile or replace QSS variables
- Apply the stylesheet
- Notify widgets when theme assets change

- Save the selected theme
- Fall back safely if a theme is missing or invalid
- Support switching themes without restarting where practical

==========

21. FUTURE THEME DEFINITIONS

==========

Prepare for the following visual directions.

HIGH FANTASY

- Stone
- Dark wood
- Brass
- Parchment
- Gemstone accents
- Warm gold headings

OLD SCROLL

- Parchment-heavy presentation
- Ink borders
- Wax seals
- Hand-drawn icons
- Faded maps
- Warm brown and cream palette

MODERN

- Flat or lightly elevated cards
- Clean geometric layout
- Minimal ornament
- Sharp typography
- Neutral surfaces
- Clear accent color
- Reduced texture

DARK FANTASY

- Blackened iron
- Worn leather
- Desaturated parchment
- Deep crimson accents
- Subtle ash or smoke texture
- Gothic but readable typography

All themes must preserve:

- Panel positions
- Navigation behavior
- Widget functionality
- Data binding
- Keyboard shortcuts

- Accessibility
- Save compatibility

==========

22. THEME SETTINGS SCREEN

==========

Add a future-ready theme selector under Settings.

It should support:

- Theme preview cards
- Theme name
- Theme description
- Theme author
- Theme version
- Apply button
- Restore default button
- Optional font-size control
- Optional interface-scale control
- Optional reduced-motion setting
- Optional high-contrast mode

Do not implement a full theme editor in the first version.

Only build the infrastructure necessary to add one later.

==========

23. RESPONSIVE AND RESIZABLE BEHAVIOR

==========

The application is primarily desktop-focused.

Target initial window:

- 1600 × 900 or similar 16:9 size

Minimum supported size:

- Approximately 1280 × 720

Behavior rules:

- Main panels resize proportionally
- Right utility sidebar keeps a practical minimum width
- Narrative area receives remaining flexible space
- Inventory slots retain a consistent square aspect ratio
- Long panels become scrollable
- Text never overlaps decorative frames
- Important controls do not disappear
- At small widths, secondary information may collapse into tabs
- User-adjusted splitter positions should be saved

Prepare for high-DPI displays.

Avoid fixed pixel assumptions wherever possible.

==========

24. ACCESSIBILITY AND READABILITY

==========

Fantasy styling must never make the application difficult to use.

Requirements:

- Body text must remain readable
- Sufficient color contrast
- Keyboard navigation
- Visible focus states
- Tooltips for icon-only buttons
- Scalable text
- Optional reduced texture
- Optional high-contrast mode later
- Do not rely only on color to communicate state
- Use icons, labels, borders, or patterns alongside color
- Resource bars must include numeric values
- Selected tabs must remain identifiable in grayscale

==========

25. INTERACTION RULES

==========

Use a central UI controller or application event system.

Widgets should emit semantic actions such as:

- open_profile
- open_inventory
- open_skill
- select_item
- use_item
- equip_item
- open_quest
- submit_player_action
- open_map
- open_journal
- change_theme
- save_game
- load_game

Widgets must not directly manipulate unrelated widgets.

The controller coordinates:

- State retrieval
- Validation
- Panel refresh
- Service calls

- Navigation
- Error presentation

==========

26. MOCK DATA REQUIREMENT

==========

Before database integration is complete, create mock data that demonstrates:

- A player portrait
- Multiple resource bars
- At least 20 inventory items
- Equipment
- At least 16 skills
- Three quests
- Active and completed objectives
- Event-log categories
- Recent-event timeline
- Map markers
- Location information
- Currency and carry weight

Mock data must be clearly separated from production services.

The UI should be replaceable with real service data without redesigning widgets.

==========

27. IMPLEMENTATION ORDER

==========

Complete the UI in controlled phases.

PHASE 1 — FOUNDATION

- Theme schema
- Theme manager
- High Fantasy theme tokens
- Main application window
- Main split layout
- Basic reusable themed panel
- Placeholder content

PHASE 2 — STATIC VERSION 1 LAYOUT

- Player summary
- Location summary
- Narrative panel
- Quest objective panel
- Right utility sidebar
- Minimap placeholder
- Utility tabs
- Event log
- Bottom navigation

PHASE 3 — DATA-DRIVEN WIDGETS

- Resource bars
- Inventory grid
- Skills grid
- Equipment slots
- Quest list
- Recent events
- Tooltips
- Selection states

PHASE 4 — INTERACTION

- Navigation
- Tab switching
- Item selection
- Context menus
- Input submission
- Splitter persistence
- Settings panel
- Theme selection

PHASE 5 — SERVICE BINDING

- Replace mock data with application services
- Bind SQLite-backed models
- Add refresh events

- Add error handling
- Add empty states
- Add loading states

PHASE 6 — THEME VALIDATION

Create simple prototype themes to prove that theming works:

- High Fantasy, fully designed
- Old Scroll, basic proof
- Modern, basic proof
- Dark Fantasy, basic proof

The three additional themes do not need final artwork yet.

They only need enough styling to prove that the same UI structure can be completely reskinned.

==========

28. CODING REQUIREMENTS

==========

- Use type hints
- Use focused modules
- Avoid one enormous MainWindow class
- Avoid repeated stylesheet strings inside widgets
- Avoid hard-coded absolute paths
- Avoid hard-coded campaign data

- Avoid hard-coded colors inside Python files
- Avoid widget-specific knowledge inside ThemeManager
- Use stable semantic object names
- Add docstrings where behavior is not obvious
- Add logging for theme-loading failures
- Add graceful fallbacks for missing assets
- Keep the application runnable after each phase
- Add tests for nonvisual logic
- Provide a screenshot or rendered preview after each major UI phase where possible

==========

29. REQUIRED TESTS

==========

Add tests for:

- Theme discovery
- Theme validation
- Missing theme fallback
- Missing asset fallback
- Token replacement
- Saved theme preference
- Inventory grid item count
- Skill grid data binding
- Quest status display
- Utility tab switching
- Navigation actions

- Resize behavior where practical
- Mock-service replacement
- No database changes from ordinary panel viewing

Visual testing may include:

- Reference screenshots
- Pixel-difference tests with reasonable tolerance
- Manual comparison checklist against the approved Version 1 image

Do not rely only on pixel-perfect tests because font rendering and operating systems may differ.

==========

30. VERSION 1 VISUAL ACCEPTANCE CRITERIA

==========

The implementation is acceptable when:

1. The overall layout clearly resembles the approved Version 1 concept.
2. The right utility sidebar remains visible during normal play.
3. Inventory, skills, profile, quests, map, and journal are accessible from the main screen.
4. The narrative panel is the primary focus.
5. The player summary is readable at a glance.
6. Active quest objectives are visible without opening a separate window.
7. The minimap is visible.
8. The interface uses original high-fantasy assets.
9. No gameplay logic depends on theme files.

10. No theme styling is hard-coded into individual widgets.

11. Switching to a prototype Modern theme changes the visual identity without changing layout behavior.

12. Resizing does not cause major overlap or unreadable content.

13. All icon-only controls have tooltips.

14. The main window can run using mock data.

15. The application remains ready for LM Studio and SQLite integration.

==========

31. WORK MODE RULES

==========

Before changing code:

1. Read the project instructions.

2. Inspect the current repository.

3. Inspect the approved Version 1 reference image.

4. Identify the current workflow step.

5. List the files that must be created or changed.

6. Do not redesign unrelated systems.

During implementation:

- Work on one phase or component at a time.

- Provide complete runnable files.

- Preserve existing working behavior.

- Add tests.

- Do not skip theme architecture to make the first screenshot faster.

- Do not place temporary styling directly inside widgets unless clearly marked and removed before completion.

At the end of each work session, report:

1. What was completed
2. Files created
3. Files modified
4. Tests run
5. Test results
6. Screenshots or preview produced
7. Known limitations
8. The single next step

Stop after the assigned step and ask whether to continue.

==========

32. FIRST ASSIGNMENT

==========

Do not build the entire interface immediately.

Begin with UI Phase 1 only:

1. Inspect the existing repository and project documents.
2. Create the proposed UI folder structure.
3. Create the theme schema.

4. Create ThemeManager.

5. Create the High Fantasy theme folder and initial theme tokens.

6. Create a minimal MainWindow containing:

   - Top header

   - Flexible left/center area

   - Fixed-minimum-width right utility sidebar

   - Bottom navigation area

7. Use placeholder labels only.

8. Demonstrate that the High Fantasy stylesheet loads.

9. Add a second minimal test theme named Modern Proof.

10. Demonstrate that switching between High Fantasy and Modern Proof changes styling without rebuilding the window.

11. Add tests for theme discovery, loading, fallback, and preference persistence.

12. Do not implement inventory, quests, skills, or LM Studio integration yet.

End with:

- Run instructions

- Test instructions

- File summary

- Screenshot or visual description

- The single next phase

- A request for approval before continuing

IMPLEMENTED APPLICATION STATUS- RELEASE 24

Last reconciled: 2026-07-23

The original phased build instructions are retained as historical guidance. The application is now at Release 24; the following describes the current implemented UI baseline and supersedes instructions to build only UI Phase 1. Full history is stored in IMPLEMENTATION_HISTORY.md.

Current theme system

Default is the fresh-install theme and follows the supplied dark-stone and bronze concept. Saved user theme choices remain authoritative.

High Fantasy and Modern Dark retain the same functional widgets and layouts. Text, selection, hover, focus, disabled, rarity, and destructive states remain visible in every theme.

Dashboard and navigation

Player Summary and Resources are consolidated. Narrative/Player Action, Quests/Objectives, Event Log/Recent Events, Location Summary, and Adventurer Tools use saved resizable splitters.

All splitter handles share the same themed visual treatment while retaining their independent orientation, bounds, and saved position.

Redundant Skills, Quests, and Campaign pages are removed. Disabled blank footer buttons preserve the six-column footer geometry; Campaign management appears in Settings.

Inventory and equipment

Inventory provides 40 slots, auto-sort, compatible drag/drop, rarity colors, persistent hover cards, item context actions, durability/uses, and quantity display.

Equipped items stay in equipment and outside inventory sorting. Invalid slot combinations are rejected.

Default-theme cards place a 42-pixel semantic SVG to the left of the established name, quantity, durability, and slot text. Hover cards use a 64-pixel icon while compact grid density remains unchanged.

Narrative and responses

The panel shows the latest 20 chronological messages, keeps the newest at the bottom, and displays the player's action before the response.

Narration is normal body text; monologue is italic at the same body size; spoken dialogue is bold and slightly larger.

Six response buttons submit immediately. Truncated response text remains available in a rectangular hover tooltip created only for the active hover and disposed when interaction ends.

Narrative styling follows the model-declared segment kind. The UI does not split passages or insert quotation marks; only transport backticks are removed.

Supporting pages and dialogs

Settings uses two invisible half-width columns with controls on the left and Campaign management on the right; the selected theme continues across the page.

People uses an independent saved splitter between conversation and its centered one-third-width trade region.

Combat rewards, quest rewards, and crafting results use theme-aware durable popups. Skills display thin theme-colored XP progress bars.

Character Creation begins with World & Safety, uses compact centered one-third-width forms, non-editable selectors, focus-skill buttons, and validated local-model randomization.

Release 25 UI requirements

Inventory and equipment cards use a horizontal icon/text layout in every shared grid instance and in the packaged executable.

Healing Potion, Greater Healing Potion, Minor Healing Vial, arrows, bandages, and restorative draughts use explicit semantic SVG mappings where a generic resolver would be ambiguous.

Hover cards are lazy top-level tooltips: create on hover, close and dispose on leave or interaction, and never instantiate one per card during page construction.

Adventure trade intent selects the current merchant on People; returning to Adventure displays the persisted conversation in Narrative & Player Action.